

***** Polymorphism *****

1. Polymorphism is a concept by which we can perform a single action in different ways. Polymorphism basically means many ways to represent a single entity or item.

***** Types Of Polymorphism *****

1. Compile Time / Static Polymorphism

Achieved by method overloading

Method overloading means methods have same name but return types, arguments and order of arguments differ.

Example: Multiple constructors Method Overloading

```
A a1 = new A(); int sum(int a, int b) { return a + b; }
```

```
A a2 = new A(3,4); int sum(int a, int b, int c) { return a + b + c; }
```

It is known as Compile Time Polymorphism because which method or constructor has to be called is decided at the compile time.

2. Run Time / Dynamic Polymorphism

Achieved by method overriding

When the type of reference variable is parent class and the object is of child class and if both parent and child contains exactly same methods then which method is called depends on object (Hence in this case the child class method's will be called). This is known as upcasting.

Although child class method is called we can not remove parent class method because by parent class method only java will know it has to access "area" method but which "area" method will be accessed on run time will be dependent on object of class method.

Ques. How Java determines which method will be called in case of method overriding?
Java decides it by Dynamic Method Dispatch.

***** Note *****

1. Reference type decides which variables/methods will be accessed

2. Object type decides which method will be run.

3. Every class is derived from the default "Object" class

4. If i create an object of any class and try to print it without defining its print method then it will call the default "toString" method of Object class and it will give some random value.

But i can "override" "toString" method of "Object" class.

3. Early Binding

If you use "final" keyword with the method then no "overriding" is possible. And hence, which method to be run will be known at compile time and it is only known as early binding.

4. Late Binding

If you are doing "overriding" then which method to be run is decided on run time and it is known as late binding.

5. You can also use "final" keyword in front of "class" to prevent "inheritance".

6. Whenever you make any class "final" implicitly it declares all of its methods final too.

7. // In both the below cases "Box" "greeting" method is called and no overriding is done because static methods are not dependent

// on objects, hence "Box" class "greeting" method is called here not "BoxWeight" object greeting method is called.

```
Box box1 = new Box(4.6, 7, 9);  
box1.greeting();
```

```
Box box2 = new BoxWeight();  
box2.greeting();
```

// Static method can be inherited but cannot be override

```
BoxWeight box = new BoxWeight();  
box.greeting();
```

8. Both overloading and overriding doesnot apply on instance variable.